Formal Execution Semantics and Safety Invariants for Governance Control Planes in Autonomous Systems

John M. Willis
Sustainable Future Tech
jwillis@sustainablefuturetech.com
ORCID: 0009-0007-5889-9628

March 25, 2026

Abstract

Autonomous AI agents are increasingly capable of executing operational actions across enterprise infrastructure, including provisioning identities, modifying infrastructure resources, and initiating financial or operational transactions. Ensuring that such actions remain aligned with enterprise governance policy introduces a fundamental challenge in the design of autonomous systems.

This paper presents a formal model of deterministic governance control planes, a class of architectures designed to enforce policy-constrained operational behavior through execution-time validation. The model separates agent reasoning from action authorization by introducing a governance control plane that evaluates proposed operational mutations against canonical system state prior to execution.

We define the execution semantics of this model and establish a set of safety properties governing policy-constrained state transitions. Specifically, we prove invariant preservation, non-bypassability of governance enforcement, deterministic evaluation consistency, execution target binding, and elimination of time-of-check/time-of-use inconsistencies under canonical state evaluation.

Using a worked example based on identity provisioning within an enterprise identity and access management domain, we illustrate how external policy sources are compiled into structured agent specifications and enforced through a deterministic governance evaluation pipeline. The resulting model provides a foundation for reproducible governance decisions, verifiable auditability, and alignment between enterprise policy frameworks and autonomous agent execution.

Keywords: Autonomous systems, AI governance, policy enforcement, control plane architectures, deterministic validation, formal methods, safety invariants, distributed systems, access control, policy compilation

Cite as: Willis, J. (2026). *Formal Execution Semantics and Safety Invariants for Governance Control Planes in Autonomous Systems*.
https://doi.org/10.5281/zenodo.19241732

1 Introduction

Autonomous AI agents are increasingly capable of executing operational actions across enterprise infrastructure, including provisioning identities, modifying infrastructure resources, and initiating financial or operational transactions. Ensuring that such actions remain aligned with enterprise governance policy introduces a fundamental challenge in the design of autonomous systems.

This paper presents a formal model of governance control planes for autonomous systems and establishes provable safety properties governing policy-constrained state transitions under deterministic validation. This work builds on prior efforts in the PBSAI governance architecture (Willis, 2026b), runtime governance control plane models (Willis, 2026c), and deterministic governance control plane implementations (Willis, 2026a). We formalize the execution semantics of governance control planes and prove correctness properties of the resulting model.

Recent advances in machine learning and agent-based systems have highlighted both the capabilities and risks associated with autonomous decision-making and tool use (Amodei et al., 2016; Bommasani et al., 2021; Hendrycks et al., 2022). Prior work has documented concerns related to unintended behavior, misalignment, and emergent risks in increasingly capable AI systems (Brundage et al., 2018; Weidinger et al., 2022). Ensuring that autonomous systems operate within defined governance constraints has therefore emerged as a central requirement for safe and trustworthy AI deployment (Russell, 2019).

In enterprise environments, autonomous agents operate over distributed systems that enforce strict requirements for correctness, reliability, and security. When such agents propose operational mutations—such as creating user identities, modifying infrastructure configurations, or updating financial records—those actions must be evaluated against organizational policy prior to execution. This requirement extends beyond traditional access control, as it involves reasoning about system state, execution context, and the consistency of authorization decisions over time.

Such systems are often structured as governed AI estates, as described in the PBSAI architecture (Willis, 2026b).

A key challenge arises from the potential divergence between the state used during policy evaluation and the state present at execution time. Prior work in system security has shown that time-of-check versus time-of-use (TOCTOU) inconsistencies can lead to race conditions and unintended system behavior (Bishop and Dilger, 1996). Addressing this issue requires an execution model in which authorization decisions are formally bound to the system state against which they are validated.

This paper formalizes a deterministic governance control plane architecture that addresses this challenge. The model separates agent reasoning from action authorization by introducing a control-plane mechanism that evaluates proposed operational mutations prior to execution. Autonomous agents generate candidate actions, which are submitted to the governance control plane for validation against a compiled representation of enterprise policy. Authorization decisions are derived deterministically from canonical system state and the structured representation of the proposed action, including its execution context.

Traditional enterprise governance frameworks rely on policy definitions expressed through organizational standards, regulatory requirements, and security frameworks. These policies are typically specified in human-readable form and are not directly executable. The approach presented in this paper introduces a governance compilation process that translates external policy sources into structured, machine-interpretable specifications. These specifications define the admissible action space, validation rules, and execution constraints governing autonomous system behavior.

The resulting model defines a control-plane execution semantics in which admissibility is derived deterministically from canonical system state.

Figure 1 illustrates the relationship between autonomous agents, governance policy sources, and the governance control plane responsible for enforcing policy-constrained execution.

The proposed architecture builds on foundational work in security policy enforcement and system design (Lampson, 1974; Saltzer and Schroeder, 1975; Schneider, 2000), while extending these principles through a formal execution model that captures the interaction between policy validation, system state, and operational

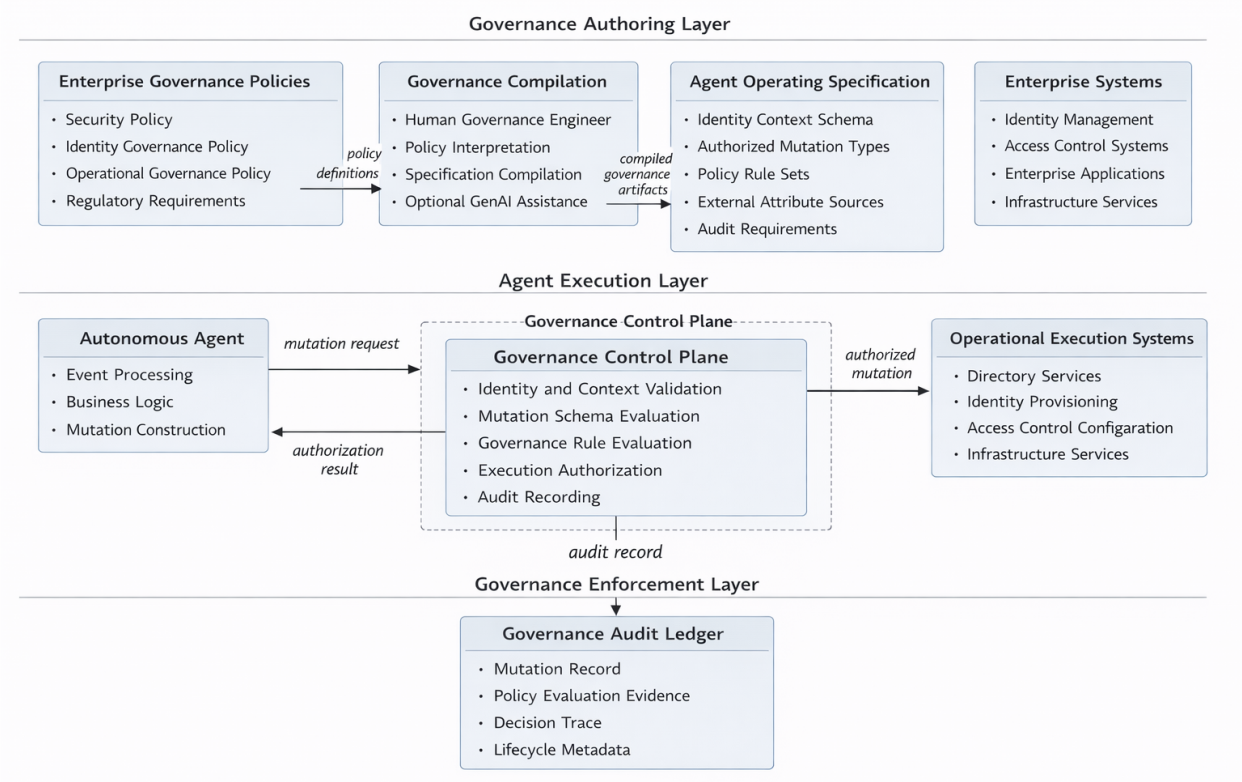

Figure 1: Governance Control Plane Architecture for Autonomous Operational Mutations

mutation.

Contributions. The contributions of this paper are fourfold:

1. A formal model of deterministic governance control planes for autonomous systems, including execution semantics for policy-constrained state transitions.

2. A deterministic validation function and execution framework that separates agent reasoning from action authorization through a governance control plane.

3. A set of formally defined and proven safety properties, including invariant preservation, non-bypassability of governance enforcement, deterministic evaluation consistency, execution context binding, and elimination of time-of-check/time-of-use inconsistencies.

4. A governance compilation model and worked example demonstrating how external enterprise policy sources are translated into structured agent operating specifications and enforced within an enterprise identity provisioning scenario.

This work does not introduce a new formalism, but instead provides a formalization of governance control plane architectures, making explicit the execution semantics and safety properties that are typically implicit in enterprise policy enforcement systems.

The contribution of this work lies not in introducing new formal verification techniques, but in structuring governance control planes as formally analyzable systems with explicit execution semantics and enforceable safety properties. In particular, the separation between compiled action constraints and state invariants,

combined with deterministic validation over canonical system state, provides a unified model that connects enterprise policy enforcement with formal system safety guarantees. This clarification addresses a gap in existing policy enforcement architectures, where execution semantics and correctness properties are typically implicit rather than formally defined.

Unlike prior work in policy engines and access control systems, which typically evaluate authorization at request time without formal guarantees over execution semantics, this work formalizes the validation and commit model and establishes provable safety properties governing policy-constrained state transitions under a formally defined execution model.

The remainder of the paper proceeds as follows. Section 2 introduces the enterprise governance context in which autonomous agents operate. Section 3 describes the governance compilation model used to translate external policy into machine-interpretable specifications. Section 4 defines the agent operating specification derived from compiled governance policy. Section 5 establishes the formal execution semantics and proves safety and correctness properties of the governance control plane model. Section 6 presents the deterministic governance evaluation pipeline and its execution semantics. Section 7 analyzes deterministic governance properties in distributed environments, and Section 8 discusses implications for autonomous system governance.

2 Enterprise Governance Context

Enterprise information systems operate within a complex governance environment defined by organizational policy, regulatory obligations, and security standards. Governance frameworks establish constraints on how operational actions may be performed, who may perform them, and under what conditions those actions are authorized. As autonomous AI systems begin to participate in operational decision-making, these governance constraints must be translated into mechanisms that can be enforced within automated execution environments.

Modern enterprise security governance is typically implemented through a combination of policy frameworks, identity management systems, and infrastructure security controls. Widely adopted standards such as the NIST Artificial Intelligence Risk Management Framework (AI RMF) emphasize the importance of governance mechanisms that ensure AI systems operate within defined organizational risk tolerances (NIST, 2023a). Similarly, enterprise identity and access management architectures commonly rely on the Digital Identity Guidelines defined in NIST Special Publication 800-63 and the broader security control catalog defined in NIST Special Publication 800-53 (NIST, 2017, 2020). These frameworks establish governance expectations for identity lifecycle management, authorization controls, and system accountability.

Within enterprise environments, identity governance plays a central role in enforcing operational policy. Role-based access control (RBAC) models are widely used to define the relationship between user identities, organizational roles, and the privileges granted to those roles (Sandhu et al., 1996; Ferraiolo et al., 2001). In RBAC systems, access permissions are derived from structured policy definitions that map organizational responsibilities to permitted system actions. Identity provisioning workflows enforce these mappings by ensuring that newly created identities are assigned privileges consistent with governance policy.

However, traditional governance mechanisms were designed for environments in which human administrators performed operational actions. In such environments, policy enforcement occurs primarily through approval workflows, manual review processes, and centralized administrative control. Autonomous AI agents introduce a new challenge: operational actions may now be proposed and executed by systems capable of generating complex reasoning chains and interacting with multiple infrastructure services.

Recent advances in agent architectures illustrate how AI systems can autonomously reason about tasks, invoke external tools, and iteratively refine their behavior based on feedback (Yao et al., 2023; Shinn et al., 2023; Park et al., 2023). While these capabilities enable powerful forms of automation, they also intro-

duce the risk that agent actions may diverge from enterprise governance policy unless explicit enforcement mechanisms are introduced.

This paper addresses that challenge by introducing a governance control plane architecture that separates agent reasoning from operational authorization. Instead of allowing agents to directly execute operational mutations, the architecture requires agents to submit proposed actions to a governance control plane that evaluates the request against compiled governance policy.

Figure 2 illustrates the relationship between external policy sources, governance compilation processes, and the resulting agent operating specification used at runtime.

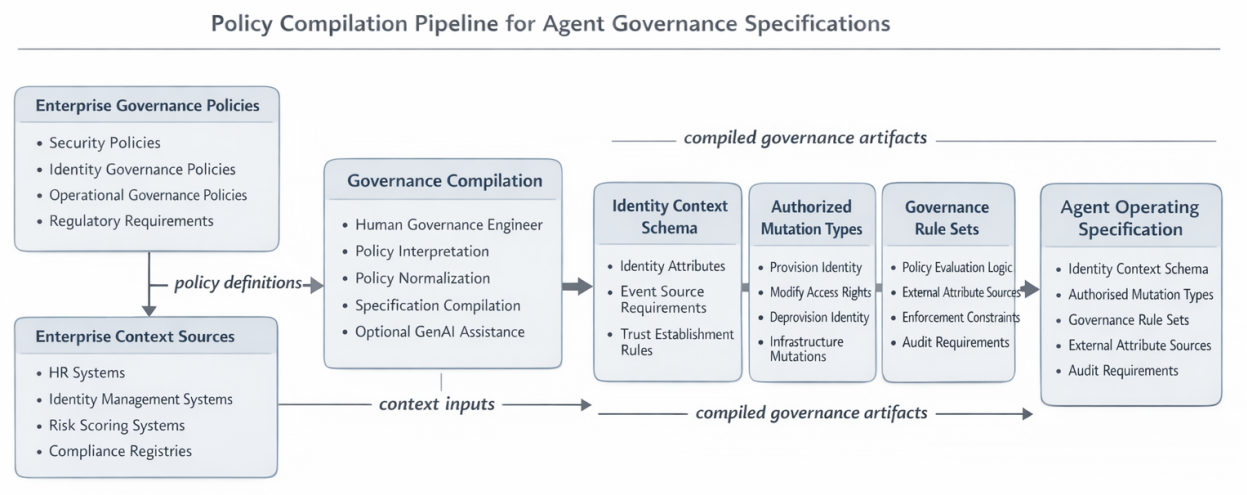

Figure 2: Governance Policy Compilation into Agent Operating Specifications

In this model, governance policy originates from multiple external sources, including organizational policy documents, regulatory requirements, identity governance frameworks, and enterprise security standards. These sources are interpreted and compiled into structured governance artifacts that define the operational constraints under which autonomous agents may operate.

The compilation process produces an agent operating specification that encodes governance constraints in machine-interpretable form. At runtime, the governance control plane evaluates agent action requests against these compiled constraints before allowing operational execution. By separating policy interpretation from runtime enforcement, the architecture ensures that enterprise governance requirements remain consistently applied even as autonomous systems perform increasingly complex operational tasks.

This governance compilation process is discussed in detail in the following section.

3 Governance Compilation Model

Enterprise governance policy typically originates in forms that are not directly usable by autonomous software systems. Organizational policy documents, regulatory requirements, and security standards are most often expressed in natural language or procedural documentation. While these sources define the authoritative constraints governing enterprise operations, they must be translated into structured machine-interpretable representations before they can be enforced within automated systems.

The process of translating external governance policy into machine-interpretable artifacts is referred to in this paper as *governance compilation*. As illustrated in Figure 2, governance compilation transforms heterogeneous policy sources into a structured specification that can be evaluated at runtime by the governance control plane.

Governance compilation is typically performed by a human policy engineer or system architect who interprets enterprise governance requirements and translates them into structured policy artifacts. In practice, this process may be assisted by software tooling or generative AI systems capable of analyzing policy documents and proposing candidate governance constraints. However, the authoritative governance specification ultimately reflects deliberate human interpretation of policy intent and organizational requirements.

The output of the governance compilation process is an *agent operating specification*. This specification defines the operational constraints under which a particular autonomous agent may act. Rather than encoding general-purpose reasoning logic, the specification describes the governance envelope within which the agent may propose operational mutations.

Within the PBSAI governance architecture, the agent operating specification functions as a domain-specific governance profile. PBSAI defines a multi-domain governance framework that organizes enterprise governance policy into a structured set of macro domains covering identity, infrastructure, data governance, operational processes, and other enterprise control areas (Willis, 2026b). Each domain may define governance constraints relevant to the operational activities performed within that domain.

In the context of this paper, the worked example focuses on the identity governance domain. Identity provisioning represents a particularly relevant scenario for autonomous system governance because identity lifecycle operations directly influence system security boundaries. Incorrect identity provisioning decisions may grant unauthorized access to enterprise systems or violate regulatory compliance requirements.

The governance compilation process therefore begins by identifying the relevant external policy sources governing identity provisioning. These sources may include organizational identity governance policy, regulatory requirements governing access control, and enterprise security standards that define how identities may be created and assigned privileges.

The compiler interprets these sources and derives a set of structured governance constraints. These constraints are then encoded within the agent operating specification in forms suitable for runtime evaluation. Examples of compiled governance artifacts may include:

- constraints on the events that may trigger agent behavior
- rules defining permissible operational mutations
- references to external systems used to derive operational attributes
- authorization boundaries governing which mutations may be proposed

Importantly, the agent operating specification does not contain the reasoning logic used by the agent to determine when actions should be proposed. Instead, the specification defines the governance constraints under which such actions may be evaluated. The autonomous agent may still perform complex reasoning processes internally, but any resulting operational mutation must pass through the governance control plane for authorization.

This separation between reasoning and authorization is a key architectural principle of the deterministic governance control plane model. By isolating governance constraints within the compiled specification and enforcing them through the control plane, the architecture ensures that operational behavior remains consistent with enterprise governance policy regardless of how agent reasoning evolves.

The next section examines the structure of the compiled agent operating specification in detail and demonstrates how external governance policy is represented within that specification.

4 Agent Operating Specification

The governance compilation process described in the previous section produces a structured artifact that defines the operational constraints under which an autonomous agent may act. This artifact is referred to as

the *agent operating specification*. The specification serves as the formal contract between the autonomous agent and the governance control plane.

The agent operating specification does not describe the internal reasoning processes used by the agent to determine when operational actions should be proposed. Instead, the specification defines the governance envelope within which those proposed actions must be evaluated. Any operational mutation generated by the agent must be submitted to the governance control plane for evaluation against the compiled constraints contained within the specification.

This architectural separation between agent reasoning and action authorization is consistent with long-standing principles of security architecture that emphasize the separation of mechanism and policy (Saltzer and Schroeder, 1975). In this model, the agent may implement complex reasoning or planning mechanisms internally, but the authorization of operational mutations remains the responsibility of the governance control plane.

Figure 3 illustrates the conceptual relationship between the autonomous agent, the agent operating specification, and the governance control plane.

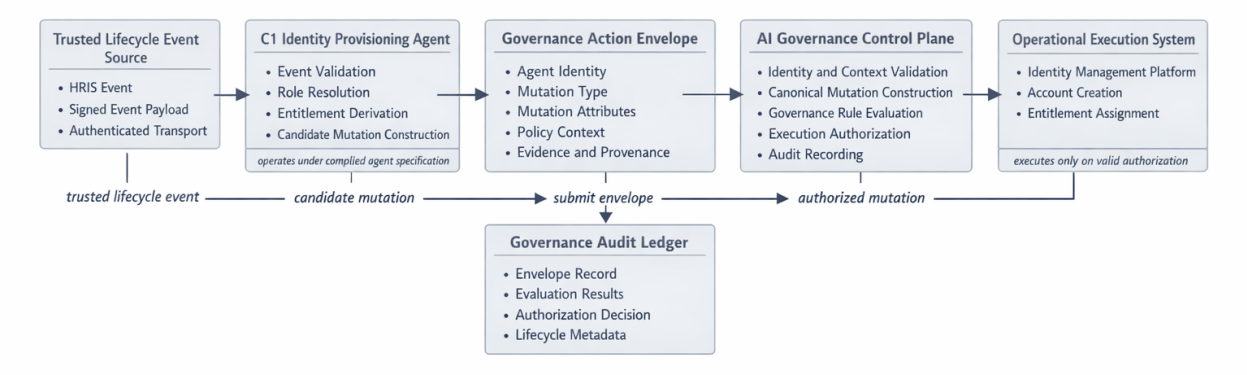

Figure 3: Agent Governance Contract Between Autonomous Agent and Governance Control Plane

The agent operating specification therefore functions as a machine-readable representation of governance policy that can be evaluated deterministically at runtime. By encoding governance constraints within a structured specification, the architecture ensures that operational actions proposed by autonomous agents remain aligned with enterprise governance policy. A formal definition of the canonical governance action envelope and its required fields is provided in Appendix A.

4.1 Overview of the Agent Operating Specification

The agent operating specification defines the governance context within which a particular autonomous agent operates. It includes a set of structured fields that describe the permissible operational behaviors of the agent, the events that may trigger those behaviors, and the external systems that may be consulted during policy evaluation.

Within the PBSAI governance architecture, each agent specification is associated with a particular governance domain and operational responsibility (Willis, 2026b). In the worked example presented in this paper, the specification corresponds to an identity provisioning agent operating within the enterprise identity governance domain.

Identity provisioning is a particularly suitable scenario for illustrating governance compilation because the operational behavior of identity management systems is already strongly governed by enterprise policy frameworks. For example, identity lifecycle operations must typically conform to organizational access control policies, regulatory compliance requirements, and internal security standards. These policies define

constraints on how identities may be created, which attributes must be assigned during provisioning, and how entitlements are derived.

The agent operating specification therefore represents the compiled interpretation of these external governance sources. Rather than allowing the agent to directly construct identity provisioning operations, the specification defines the structure of the actions that may be proposed and the governance rules under which those actions will be evaluated.

In practice, the specification includes multiple categories of governance information. These include definitions of the event sources that may trigger agent execution, schemas describing the structure of operational mutation requests, references to external systems that provide authoritative identity information, and rule sets that determine whether a proposed mutation may be authorized.

Importantly, the specification does not encode the logic used by the agent to derive entitlements or determine when provisioning actions should occur. Those behaviors may be implemented within the agent's internal reasoning mechanisms. The governance control plane remains agnostic to those reasoning processes. Instead, the control plane evaluates the resulting operational mutation request against the compiled governance constraints defined within the specification.

The following subsections describe the specific components of the agent operating specification and illustrate how each component reflects the compilation of external governance policy.

4.2 Event Source Definitions

The first component of the agent operating specification defines the event sources that may trigger agent execution. Autonomous agents operating within enterprise environments typically respond to operational events generated by upstream systems. These events represent state changes occurring within authoritative enterprise platforms such as human resources systems, infrastructure management systems, or application lifecycle management platforms.

In the identity provisioning scenario examined in this paper, the primary triggering event originates from the enterprise human resources information system (HRIS). When a new employee record is created within the HRIS, an event is generated indicating that an identity provisioning workflow should be initiated.

However, in order for such an event to be trusted by the governance control plane, the system that generated the event must itself be authenticated and authorized as a legitimate event source. Simply receiving an event claiming to originate from the HRIS is insufficient. The governance architecture must be able to verify that the event was produced by the authoritative system responsible for maintaining employee records.

The agent operating specification therefore contains a compiled definition of trusted event sources. This definition is derived from enterprise identity governance policy that identifies which systems are authoritative for particular classes of operational events. In the identity provisioning domain, enterprise policy typically establishes the HRIS as the system of record for employee lifecycle events. The governance compiler interprets this policy and encodes it into the specification as a trusted event source.

A simplified example of the compiled event source definition appears below:

```
event_sources:
- source_id: HRIS
system_type: authoritative_identity_system
event_type: employee_created
authentication_method: mTLS
trust_anchor: enterprise_ca
allowed_topics:
- hr.employee.created
```

Each field in this structure reflects a specific governance decision derived from external policy sources.

Source Identifier. The `source_id` field uniquely identifies the system authorized to produce the event. This identifier is typically aligned with enterprise configuration management records or identity system registries.

System Type. The `system_type` field indicates the governance role of the event source. In this case, the HRIS is designated as an *authoritative identity system*, meaning that employee lifecycle information produced by this system is considered authoritative for identity provisioning operations.

Event Type. The `event_type` field defines the specific class of event that may trigger the agent. This constraint ensures that the agent is only invoked for events relevant to the provisioning workflow.

Authentication Method. The `authentication_method` field specifies how the governance control plane verifies the identity of the event source. In many enterprise environments, this verification occurs through mutual Transport Layer Security (mTLS), where both the event producer and the receiving system authenticate using cryptographic certificates.

Trust Anchor. The `trust_anchor` field identifies the certificate authority used to validate the identity of the event-producing system. Only systems presenting certificates issued by the enterprise trust anchor are permitted to generate trusted events.

Allowed Topics. The `allowed_topics` field restricts the messaging channels through which the event may be delivered. This prevents malicious systems from injecting events through unauthorized communication channels.

Together, these fields ensure that the governance control plane can verify the authenticity and provenance of the event that triggered agent execution. The importance of maintaining verifiable event ordering and trusted system coordination has long been recognized in distributed systems research, where reliable event sequencing is necessary for maintaining consistent system state across distributed components (Lamport, 1978).

Once an event is received and validated against the compiled event source definition, the governance control plane proceeds to the next stage of evaluation, which involves constructing the operational mutation envelope submitted by the agent.

4.3 Operational Mutation Schema

Once a trusted event has been validated, the autonomous agent may propose an operational mutation corresponding to the requested action. In the identity provisioning scenario examined in this paper, the operational mutation corresponds to the creation of a new enterprise identity within the identity management system.

However, the agent cannot propose arbitrary operational actions. The structure of any proposed mutation must conform to a schema defined within the compiled agent operating specification. This schema ensures that the governance control plane receives mutation requests in a deterministic and machine-interpretable form.

The operational mutation schema is therefore a key artifact produced by the governance compilation process. The schema defines the set of fields that must be present in the mutation request, the allowed data types for each field, and the governance semantics associated with those fields.

A simplified example of a compiled operational mutation schema appears below:

```
mutation_schema:
mutation_type: identity.create
required_fields:
- employee_id
- legal_name
- department
```

```
- job_code
- manager_id
- employment_type
- execution_target
optional_fields:
- location
- cost_center
- contractor_flag
```

The `execution_target` field specifies the system or component responsible for applying the authorized operational mutation. By including the execution target within the mutation schema, the governance model ensures that authorization decisions are bound both to the proposed action and to the system context in which that action will be executed. This prevents post-authorization redirection of an approved mutation to a different execution destination.

Each field included in the schema corresponds to information required by enterprise identity governance policy. These requirements typically originate from multiple external sources, including identity governance frameworks, regulatory compliance obligations, and internal organizational policy.

For example, enterprise identity governance policy may require that all identities be associated with an authoritative employee identifier, a responsible manager, and an organizational department. These requirements are interpreted during governance compilation and encoded into the mutation schema as required fields.

The schema therefore functions as a structural contract between the agent and the governance control plane, defining both the structure of permissible mutations and the authorized execution context in which those mutations may be applied. Any mutation request submitted by the agent must conform to the schema defined within the specification. Requests that do not conform to the schema are rejected prior to policy evaluation.

It is important to emphasize that the mutation schema defines the *structure* of permissible operational mutations, but it does not implement the logic used to derive the values of those fields. The responsibility for constructing a valid mutation request remains within the agent implementation.

For example, the agent may query external systems to obtain employee attributes, derive entitlement candidates based on job codes, or perform reasoning processes to determine whether provisioning should occur. These behaviors are part of the agent's internal implementation and are not specified within the governance specification.

The governance control plane remains agnostic to the reasoning processes used by the agent. Its responsibility is limited to evaluating the resulting mutation request, including its designated execution target, against the compiled governance constraints.

This separation between reasoning and authorization ensures that governance policy remains enforceable even as agent capabilities evolve. Regardless of how the agent derives the mutation request, the request must conform to the schema defined within the specification before it can be considered for authorization.

After the mutation request has been validated against the schema, the governance control plane constructs a canonical mutation representation that can be evaluated deterministically within the governance pipeline. This canonicalization process is described in the following subsection.

4.4 External Attribute Resolution

Operational mutations proposed by autonomous agents frequently depend on information obtained from external enterprise systems. In the identity provisioning scenario examined in this paper, attributes such as department, manager relationships, employment type, and entitlement assignments are not generated by

the agent itself. Instead, these attributes originate from authoritative enterprise systems that maintain the underlying business records.

The agent operating specification therefore includes definitions of the external attribute sources that may be consulted during mutation construction. These definitions are derived during governance compilation from enterprise architecture documentation and identity governance policy that identifies authoritative systems of record.

In most enterprise environments, employee lifecycle information originates from the human resources information system (HRIS), while entitlement information is derived from identity management or access governance platforms. Governance policy typically defines these systems as the authoritative sources for their respective data domains.

A simplified example of the compiled attribute source definitions appears below:

```
attribute_sources:
employee_attributes:
source_system: HRIS
attributes:
- employee_id
- legal_name
- department
- manager_id
- job_code
- employment_type

entitlement_catalog:
source_system: IDM
attributes:
- role_definitions
- entitlement_mappings
- segregation_of_duty_rules
```

These definitions specify the authoritative systems from which relevant attributes may be obtained. The specification does not describe how the agent queries these systems or how the data retrieval is implemented. Instead, it defines the governance boundary conditions under which attribute resolution may occur.

For example, enterprise identity governance policy may require that all entitlement assignments originate from the identity management system's approved role catalog. The governance compiler interprets this policy and encodes the requirement within the agent specification by declaring the identity management system as the authoritative entitlement source.

During runtime operation, the agent may query the HRIS to obtain employee attributes associated with the triggering event. It may also query the identity management system to retrieve role definitions or entitlement mappings associated with the employee's job code or department.

However, the governance control plane itself does not perform these queries. Its responsibility is limited to evaluating the operational mutation proposed by the agent. The agent remains responsible for constructing the mutation request by retrieving information from the declared external systems.

This separation again reflects the architectural distinction between agent reasoning and governance enforcement. The agent may perform arbitrary internal logic to derive candidate entitlement assignments, but the resulting mutation must still be evaluated against the governance rules defined within the specification.

The following subsection describes how governance rules derived from external policy sources are compiled into rule sets that determine whether a proposed mutation may be authorized.

4.5 Governance Rule Compilation

The previous subsections described how the agent operating specification defines trusted event sources, the structure of permissible mutation requests, and the authoritative systems from which attributes may be obtained. The final component of the compiled specification defines the governance rules that determine whether a proposed mutation may be authorized.

Governance rules represent the compiled interpretation of external policy sources that govern operational behavior within the enterprise environment. These sources may include organizational security policy, regulatory compliance requirements, access control frameworks, and internal operational standards.

Enterprise identity governance policies frequently define rules governing identity lifecycle operations such as account creation, role assignment, and entitlement management. These policies may be expressed in a variety of forms, including written organizational policy documents, identity governance frameworks, or regulatory compliance guidance.

During governance compilation, the policy engineer interprets these external policy sources and derives a set of structured governance constraints that can be evaluated programmatically. These constraints are then encoded within the agent operating specification as rule sets associated with the operational mutation schema.

A simplified example of compiled governance rules for identity provisioning appears below:

```
governance_rules:

- rule_id: verify_authoritative_source
description: Provisioning events must originate from HRIS
condition:
event_source == "HRIS"

- rule_id: verify_required_attributes
description: Required attributes must be present
condition:
employee_id != null AND
department != null AND
manager_id != null

- rule_id: verify_role_mapping
description: Entitlements must derive from approved role mappings
condition:
role_mapping_source == "IDM"

- rule_id: segregation_of_duties_check
description: Assigned entitlements must not violate SoD constraints
condition:
not violates_sod(entitlements)
```

Each rule in this structure corresponds to a governance constraint derived from enterprise policy.

The rule `verify_authoritative_source` enforces the organizational policy that employee lifecycle events must originate from the HRIS, which serves as the authoritative identity source.

The rule `verify_required_attributes` enforces identity data quality requirements commonly specified within identity governance frameworks.

The rule `verify_role_mapping` enforces the policy that entitlements must be derived from approved role definitions maintained within the identity management platform.

The rule `segregation_of_duties_check` represents a governance constraint frequently used within enterprise access control systems to prevent conflicting privilege assignments.

These types of constraints are consistent with long-established access control models such as role-based access control (RBAC), which define how privileges are assigned based on organizational roles and governance policy (Sandhu et al., 1996; Ferraiolo et al., 2001).

Importantly, the governance control plane does not interpret human-readable policy documents at runtime. Instead, the governance compiler performs the translation from policy language into structured rule sets that can be evaluated deterministically during execution.

These constraints can be understood as compiled governance artifacts derived from higher-level policy specifications, consistent with runtime governance architectures (Willis, 2026c).

This approach ensures that runtime governance evaluation remains computationally efficient while preserving alignment with the organization's external policy framework.

Once the governance rules have been compiled into the agent operating specification, the specification contains all of the information necessary for runtime governance evaluation. The following section defines the formal execution semantics of the governance control plane model and establishes key safety and correctness properties governing policy-constrained state transitions under deterministic evaluation.

5 Formal Properties and Safety Guarantees

This section defines the formal execution semantics of the governance control plane model and establishes key safety and correctness properties governing policy-constrained state transitions under deterministic evaluation, consistent with governance control plane architectures (Willis, 2026a).

While several of the results follow directly from the structure of the model, their purpose is to make explicit and verifiable the safety guarantees that are otherwise assumed but not formally articulated in existing governance and policy enforcement architectures.

5.1 Preliminaries

Let S denote the canonical system state, and let A denote a proposed action represented as a structured mutation over S, including its execution target.

This formulation is consistent with governance control plane models in which system behavior is defined over canonical state representations and structured actions subject to policy constraints (Willis, 2026a).

Let $T(S, A)$ denote the state transition function such that:

$$S' = T(S, A) \tag{1}$$

In governance control plane architectures, such transitions are subject to deterministic evaluation over policy, constraints, and invariant conditions prior to commit (Willis, 2026a).

where S' is the resulting system state obtained by applying A to S.

In such models, execution authority is derived from canonical system state and policy evaluation rather than stored as mutable authorization state (Willis, 2026a).

Let $V(S, A) \in \{0, 1\}$ be a deterministic validation function.

Let $C(S, A)$ denote the commit authorization function such that:

$$C(S, A) = 1 \iff V(S, A) = 1 \tag{2}$$

Let $I = \{i_1, i_2, \ldots, i_n\}$ be a set of system invariants, where each invariant $i : S \to \{0, 1\}$ defines a constraint over admissible system states. These invariants are enforced in a manner consistent with governance control plane models that ensure traceability, provenance verification, and human authorization prior to state transition (Willis, 2026a).

In addition to the invariant set I, the governance compilation process produces an agent-specific constraint set, denoted $C_{\text{spec}}(A)$, which restricts the admissible structure and context of proposed actions A.

The validation function $V(S, A)$ enforces both:

- the compiled constraint set $C_{\text{spec}}(A)$ over the proposed action, and
- the invariant set I over the resulting state $S' = T(S, A)$.

Since $C(S, A) = 1 \iff V(S, A) = 1$, validation and authorization are equivalent in this model and are treated as a single decision step.

Formally, validation satisfies:

$$V(S, A) = 1 \implies C_{\text{spec}}(A) = 1 \ \wedge \ \forall i \in I, \ i(T(S, A)) = 1 \tag{3}$$

A state S is said to be valid if:

$$\forall i \in I, \ i(S) = 1 \tag{4}$$

We denote by A_{adv} an action generated under adversarial or compromised conditions.

Separation of Action Constraints and State Invariants The invariant set I is distinct from the constraint set derived from governance compilation. The compiled agent operating specification defines constraints over the admissibility, structure, and context of proposed actions A, represented by $C_{\text{spec}}(A)$. In contrast, the invariant set I defines constraints over permissible system states and resulting state transitions. The validation function $V(S, A)$ therefore operates across two layers: it enforces action-level constraints to ensure that proposed mutations are well-formed and policy-compliant, and it enforces state-level invariants to ensure that the resulting transition $S' = T(S, A)$ preserves system correctness. This separation ensures that restricting the action space alone is insufficient; admissible actions must also produce valid system states.

We assume that the validation function $V(S, A)$ is both sound and complete with respect to $C_{\text{spec}}(A)$ and I, such that:

$$V(S, A) = 1 \iff C_{\text{spec}}(A) = 1 \ \wedge \ \forall i \in I, \ i(T(S, A)) = 1$$

5.2 Discussion of Assumptions

The formal results presented in this section rely on several assumptions regarding the behavior of the governance control plane, the validation function V, and the state transition function T.

First, consistent with the formal definition in Section 5.1, it is assumed that the validation function $V(S, A)$ is both sound and complete with respect to the compiled constraint set $C_{\text{spec}}(A)$ and the invariant set I. That is, all governance-relevant constraints over action admissibility are represented in C_{spec}, and all constraints over permissible system states are represented in I, and both are enforced during validation of the transition $T(S, A)$.

Second, the model assumes that all state transitions are mediated exclusively through the governance control plane, and that no external mechanisms can directly modify the canonical system state S.

Third, the execution model assumes that validation and authorization occur over a consistent view of S, and that no intervening state transitions occur between validation and execution unless explicitly revalidated.

These assumptions are consistent with standard control-plane architectures in distributed systems, where correctness guarantees depend on the integrity of the control boundary and the completeness of policy enforcement. In practical implementations, these conditions may be approximated through mechanisms such as transactional state access, validation completeness checks, and enforcement of execution isolation.

The results presented here therefore characterize the correctness properties of the model under these architectural conditions, where action admissibility and state validity are enforced jointly by the governance control plane.

5.3 Adversarial and Trust Model

The formal results presented in this section are evaluated relative to an explicit trust and adversarial model.

The trusted computing base consists of the governance control plane, the policy registry storing compiled agent operating specifications, the canonical state source used during validation, and the mechanisms used to record governance audit artifacts. These components are assumed to operate correctly and to preserve the integrity of validation, authorization, and audit recording.

The adversary may control or influence autonomous agents, mutation contents, upstream event payloads, and external enterprise systems not included in the trusted computing base. In particular, the adversary may attempt to submit malformed mutation requests, omit required attributes, falsify mutation context, alter the designated execution target, replay previously valid mutation proposals, or induce agents to generate mutations that violate governance policy.

Adversarial strategies may be categorized as follows:

- **Mutation Integrity Attacks:** Attempts to alter mutation attributes, omit required fields, or inject malformed data into the mutation request.
- **Context Manipulation Attacks:** Attempts to falsify event source, policy context, or supporting evidence used during validation.
- **Execution Target Substitution:** Attempts to redirect an authorized mutation to a different execution target than the one evaluated during validation.
- **Replay Attacks:** Reuse of previously valid mutation proposals in a different context or at a different time.
- **Agent Compromise:** Generation of adversarial mutation proposals by a compromised or malicious autonomous agent.

The governance control plane is designed to mitigate these classes of attacks through schema enforcement, deterministic validation, execution context binding, and evaluation against canonical system state.

The governance control plane is designed to prevent unauthorized state transitions—even when the proposing agent is faulty, compromised, or strategically adversarial—provided that the assumptions defined in Section 5.2 hold, including exclusive mediation by the control plane and validation against canonical system state. Under these conditions, validation induces a strict partition over proposed mutations: if $V(S, A) = 0$, the mutation is rejected and no state transition occurs; if $V(S, A) = 1$, the mutation is authorized and produces a transition, with $V(S, A) = 1$ ensuring that both $C_{\text{spec}}(A)$ and $\forall i \in I,\ i(T(S, A)) = 1$ hold.

This model does not assume that autonomous agents are trustworthy. Instead, trust is placed in the deterministic enforcement boundary implemented by the governance control plane. The primary security objective is therefore not to ensure correct agent reasoning, but to ensure that incorrect, manipulated, or maliciously generated mutation requests cannot produce unauthorized state transitions.

The model does not address adversaries capable of compromising the governance control plane itself, corrupting the canonical state source, tampering with the policy registry, or modifying audit records after they are written. Such attacks fall outside the scope of the formal guarantees established in this work and would require additional mechanisms such as replicated trust anchors, tamper-evident ledgers, remote attestation, or distributed consensus protocols.

5.4 Limitations

The formal model presented in this section abstracts several aspects of real-world system behavior in order to establish clear execution semantics and provable safety properties.

First, the model assumes a consistent and authoritative view of canonical system state S at validation time, and does not explicitly model concurrent state updates or distributed consistency protocols governing $T(S, A)$. In practical distributed environments, additional coordination mechanisms may be required to approximate this assumption.

Second, the model assumes that governance constraints can be cleanly separated into two components: an action-level constraint set $C_{\text{spec}}(A)$ derived from governance compilation, and a state-level invariant set I defined over admissible system states. In practice, the boundary between action admissibility and state validity may be more complex, and some governance policies may span both layers or depend on contextual information not captured explicitly in either representation.

Third, the model assumes that both the compiled constraint set C_{spec} and the invariant set I are complete with respect to their respective domains. In practice, policy incompleteness or mis-specification may lead to gaps between intended and enforced governance behavior, either through insufficient restriction of admissible actions or incomplete characterization of valid system states.

Furthermore, the model treats validation as a deterministic function over structured inputs (S, A), and does not account for probabilistic or non-deterministic components that may arise in systems incorporating machine learning-based reasoning. While the governance control plane enforces deterministic authorization, upstream agent behavior may still introduce variability in proposed actions.

Finally, the model does not explicitly address performance considerations such as latency, throughput, or scalability of the governance evaluation pipeline or the execution of $T(S, A)$. These factors are implementation-dependent and may influence the feasibility of applying the model in high-frequency or large-scale operational environments.

These limitations highlight the distinction between the formal execution model and its realization in production systems, and suggest directions for future work in extending the model to distributed, concurrent, and performance-constrained environments.

The goal of this formalization is to establish a clear and analyzable execution model for governance control planes; evaluating practical implementations, performance characteristics, and partial or distributed enforcement scenarios is left to future work.

5.5 Limit of Guarantee

The formal guarantees established in this section are conditioned on the integrity of the governance control plane and on the assumptions defined in Section 5.2.

Specifically, the results assume that:

- All state transitions are mediated exclusively through the governance control plane.
- The validation function $V(S, A)$ correctly enforces both the compiled constraint set $C_{\text{spec}}(A)$ over proposed actions and the invariant set I over resulting system states.
- The canonical system state S used during validation accurately reflects the true system state at commit time.
- The policy registry and compiled specifications defining C_{spec} are not corrupted or adversarially modified.

Under these conditions, the model guarantees that no unauthorized state transition can occur. In particular, a transition $S' = T(S, A)$ may occur only if $V(S, A) = 1$, which implies that the proposed action satisfies the compiled constraints and the resulting state satisfies all invariants:

$$V(S, A) = 1 \implies C_{\text{spec}}(A) = 1 \ \wedge \ \forall i \in I, \ i(T(S, A)) = 1 \tag{5}$$

Violations of these assumptions—such as compromise of the control plane, corruption of canonical state, incomplete or incorrect specification of C_{spec} or I, or failure of validation enforcement—may invalidate these guarantees.

The results presented in this work therefore characterize the security and correctness properties of the governance model within a trusted control-plane boundary, where both action admissibility and state validity are enforced deterministically, rather than providing end-to-end guarantees under arbitrary system compromise.

5.6 Formal Verification Perspective

Having defined the execution semantics and safety properties of the model, we now consider how the model may be interpreted within standard formal verification frameworks such as temporal logic-based model checking systems or specification languages such as TLA+.

The model can be interpreted as a guarded transition system in which compiled action constraints restrict admissible inputs and invariants constrain the reachable state space.

In such a representation, the model can be viewed as a constrained transition system with two distinct classes of constraints:

- The canonical system state S corresponds to the system state variable.
- The transition function $T(S, A)$ defines the state transition relation.
- The invariant set I corresponds to safety properties that must hold over all reachable system states.
- The compiled constraint set $C_{\text{spec}}(A)$ defines admissibility conditions over proposed actions, constraining the transition relation by restricting the set of allowable inputs.
- The validation function $V(S, A)$ defines a guarded transition relation that enforces both action-level admissibility and state-level safety.

Under this formulation, the governance control plane enforces a constrained transition system in which admissible transitions must satisfy both:

$$C_{\text{spec}}(A) = 1 \quad \text{(action admissibility)} \tag{6}$$

and

$$\forall i \in I, \ i(T(S, A)) = 1 \quad \text{(state validity)} \tag{7}$$

The resulting model corresponds to a safety-preserving transition system in which the reachable state space is restricted both by the admissible action space and by invariant-preserving transitions. This can be interpreted as a two-layer verification structure, where input constraints restrict the transition relation and invariants constrain the reachable states.

This formulation suggests that the governance model may be amenable to formal verification using model checking or theorem proving techniques, where:

- $C_{\text{spec}}(A)$ can be encoded as transition guards or preconditions, and
- I can be encoded as safety properties over the reachable state space.

Verification of such a system would involve demonstrating that all transitions permitted by the guarded transition relation preserve the invariant set I, and that no unauthorized transitions exist outside this constrained transition system.

This represents a direction for future work in applying formal verification techniques to governance control plane architectures.

The contribution of this work is therefore not in the introduction of new verification techniques, but in structuring governance control planes in a form that admits formal reasoning and verification.

The purpose of this formulation is not to claim novelty in formal verification theory, but to demonstrate that governance control plane architectures can be expressed in a form amenable to standard verification techniques. By making the transition structure, admissibility constraints, and invariant preservation conditions explicit, the model enables direct encoding within existing formal methods frameworks. Full mechanized verification of specific governance specifications remains an important direction for future work.

The following results should be interpreted within the assumptions and trust boundary defined in Sections 5.2 through 5.5.

5.7 Validation Completeness

Lemma 1 (Validation Completeness Implies Invariant Closure). If the validation function $V(S, A)$ enforces both the compiled constraint set $C_{\text{spec}}(A)$ and the invariant set I over the state transition $S' = T(S, A)$, then any action A such that $V(S, A) = 1$ produces a state S' that satisfies all invariants in I.

Proof. By assumption, $V(S, A)$ enforces both action-level constraints and state-level invariants. Therefore, if $V(S, A) = 1$, we have:

$$C_{\text{spec}}(A) = 1 \tag{8}$$

and

$$\forall i \in I,\ i(T(S, A)) = 1 \tag{9}$$

Let $S' = T(S, A)$. Then:

$$\forall i \in I,\ i(S') = 1 \tag{10}$$

Thus, the transition preserves all invariants in I, and the resulting state S' is valid.

□

5.8 Invariant Preservation

Theorem 1 (Invariant Preservation). Let S be a valid state. If $V(S, A) = 1$ and $S' = T(S, A)$, then S' is valid.

Proof. From Lemma 1, any action A such that $V(S, A) = 1$ produces a state $S' = T(S, A)$ satisfying:

$$\forall i \in I,\ i(S') = 1 \tag{11}$$

Since S is valid and all invariants in I are preserved under the transition $T(S, A)$, it follows that S' is valid.

□

5.9 Non-Bypassability of Governance Enforcement

Theorem 2 (Non-Bypassability). No state transition $S \to S'$ can occur unless there exists an action A such that $S' = T(S, A)$ and $V(S, A) = 1$.

Proof. By system constraints (Section 4), all state transitions must be mediated by the governance control plane. Therefore, any transition $S \to S'$ must occur through some action A such that:

$$S' = T(S, A) \tag{12}$$

and

$$C(S, A) = 1 \tag{13}$$

By definition of C:

$$C(S, A) = 1 \iff V(S, A) = 1 \tag{14}$$

Thus, any realized transition must satisfy $V(S, A) = 1$. Since direct modification of S outside the control plane is disallowed, no transition can occur without successful validation.

□

This property implies that no adversarial or compromised agent can induce an unauthorized state transition, provided that execution remains mediated by the governance control plane.

Corollary 1 (Adversarial Mutation Containment). For any adversarially generated action A_{adv}, a state transition $S' = T(S, A_{\text{adv}})$ can occur only if $V(S, A_{\text{adv}}) = 1$.

Proof. This follows directly from Theorem 2, which establishes that no state transition may occur unless it is mediated by the governance control plane and satisfies $V(S, A) = 1$. Since A_{adv} is simply an action proposed under adversarial conditions, it is subject to the same constraint.

□

5.10 Deterministic Evaluation Consistency

Theorem 3 (Deterministic Evaluation Consistency). For any canonical state S and action A, repeated evaluations of $V(S, A)$ yield identical results, and equivalent action representations produce identical validation outcomes.

Proof. By definition, V is a deterministic function over its inputs (S, A). Therefore, for any fixed (S, A):

$$V(S, A) = c \tag{15}$$

for some constant $c \in \{0, 1\}$. Re-evaluating V with identical inputs yields the same output. Canonicalization ensures that semantically equivalent actions correspond to identical representations of A, thereby ensuring consistent evaluation outcomes.

□

5.11 Execution Context Binding

Theorem 4 (Execution Target Binding). Authorization decisions are bound to the execution target specified in A. If the execution target is modified, the authorization decision does not necessarily hold for the resulting transition.

Proof. Let $A = (a, t)$ where t is the execution target, and consider a modified action $A' = (a, t')$ with $t' \neq t$. The transitions induced by these actions are:

$$S' = T(S, A), \quad S'' = T(S, A') \tag{16}$$

Since V evaluates the full structure of A, including the execution target, it follows that:

$$V(S, A) \neq V(S, A') \quad \text{in general} \tag{17}$$

Thus, an authorization decision for A does not imply authorization for A', and authorization is therefore bound to the execution context.

□

This property prevents an adversary from reusing an authorization decision for one execution context in order to redirect the mutation to a different execution target.

5.12 Time-of-Check / Time-of-Use Safety

Theorem 5 (TOCTOU Elimination under Canonical State Evaluation). If validation is performed against canonical state S at commit time and no intermediate state transitions occur between validation and execution, then time-of-check/time-of-use inconsistencies are eliminated.

Proof. TOCTOU inconsistencies arise when validation occurs on a state S_t and execution applies a transition $T(S_{t+\Delta}, A)$ where $S_{t+\Delta} \neq S_t$.

In this model, validation and authorization are performed at commit time against canonical state S, and execution applies the transition $S' = T(S, A)$ without reuse of prior decisions. Under the assumption that no intervening transitions occur, we have:

$$S_{\text{validation}} = S_{\text{execution}} \tag{18}$$

Thus, validation and execution operate over the same state, eliminating TOCTOU inconsistencies.

□

These results provide a formal foundation for the design of governance control planes in practical enterprise systems, where deterministic enforcement and auditability are required for safe autonomous operation.

The following section examines how the governance control plane evaluates mutation requests against the compiled specification.

6 Governance Control Plane Execution

Given the formal execution model defined in Section 5, the governance control plane realizes these semantics through a structured evaluation pipeline that evaluates operational mutations proposed by autonomous agents. The governance control plane acts as an independent authorization layer that determines whether a proposed mutation is permitted according to the compiled governance specification.

The governance control plane does not perform the operational mutation itself. Instead, it evaluates mutation proposals and returns an authorization decision to the requesting agent. If the mutation is approved, the agent may proceed with executing the corresponding operational action within the target enterprise system.

To ensure deterministic behavior, mutation evaluation occurs through a structured control-plane pipeline. Each stage of the pipeline performs a specific validation or transformation of the mutation request. This design ensures that governance evaluation remains predictable, auditable, and reproducible across distributed systems.

The pipeline used in this architecture consists of five stages:

1. Identity and Context Validation

2. Action Schema Evaluation
3. Governance Rule Evaluation
4. Commit Authorization
5. Audit and Lifecycle Recording

Each stage corresponds to a distinct responsibility within the governance evaluation process. By separating these responsibilities, the architecture ensures that governance policy can be evaluated consistently regardless of the reasoning mechanisms used by the agent.

This staged execution model reflects well-established principles of transaction processing and distributed system coordination, where complex operations are decomposed into deterministic processing stages in order to maintain consistent system state across distributed components (Gray and Reuter, 1993; Kleppmann, 2017).

While this work focuses on the formal execution semantics and correctness properties of the governance control plane, the architecture described in Section 6 directly corresponds to implementable control-plane patterns used in enterprise systems. A reference implementation aligned with the AGCP specification is currently under development to evaluate operational characteristics such as latency, throughput, and policy evaluation overhead. These implementation concerns are orthogonal to the formal safety guarantees established here, but represent an important direction for validating the model in production environments.

Figure 4 illustrates the governance control plane execution pipeline.

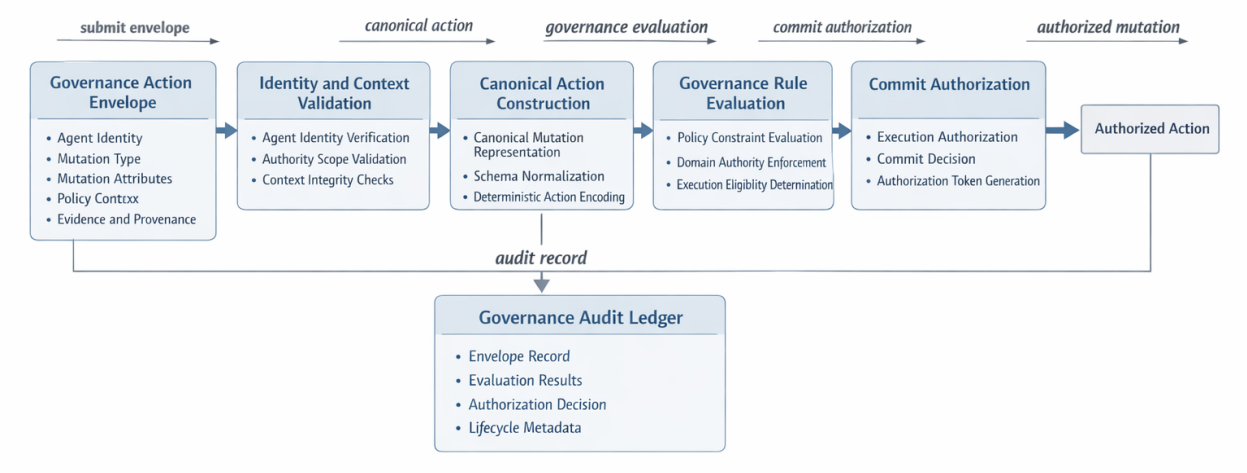

Figure 4: Governance Control Plane Evaluation Pipeline

6.1 Identity and Context Validation

The first stage of the governance pipeline verifies the identity and provenance of the mutation request submitted by the autonomous agent. Before any governance evaluation can occur, the control plane must establish that the request originated from a trusted agent operating within an approved governance domain.

This validation step includes several checks.

First, the identity of the requesting agent is verified using the authentication mechanism defined within the enterprise control plane infrastructure. In many environments, this verification occurs through cryptographic identity mechanisms such as mutual Transport Layer Security (mTLS) or signed service identities issued by the enterprise identity provider.

Second, the governance control plane verifies that the agent is authorized to operate under the agent operating specification associated with the mutation request. Each mutation request therefore includes metadata identifying the agent specification under which the mutation is proposed.

A simplified representation of the mutation request envelope appears below.

```
mutation_request:

agent_id: C1_identity_provisioning_agent
agent_spec_id: PBSAI.ICAM.C1.v1
event_source: HRIS
mutation_type: identity.create

payload:
employee_id: 483920
department: finance
manager_id: 77341
```

The governance control plane verifies that the agent identifier corresponds to a registered autonomous agent and that the referenced specification is a valid compiled governance profile. This ensures that the mutation request is evaluated against the correct governance rules.

Finally, the pipeline verifies that the event associated with the mutation request originated from a trusted event source defined within the agent operating specification. As described earlier in the event source definitions, trusted event sources are established through authenticated system identities and enterprise trust anchors.

Maintaining verifiable ordering and authenticity of events is essential in distributed systems environments where multiple components may produce asynchronous operational events. Mechanisms for establishing reliable event ordering and causality relationships have long been recognized as critical components of distributed system coordination (Lamport, 1978).

Once the mutation request has passed identity and context validation, the governance control plane proceeds to evaluate the structure of the mutation request against the compiled mutation schema.

6.2 Action Schema Evaluation

After the mutation request has passed identity and context validation, the governance control plane evaluates the structure of the proposed mutation against the compiled mutation schema defined within the agent operating specification.

This step ensures that the mutation request conforms to the structural contract defined during governance compilation. Because the schema was derived from external governance policy and encoded into the specification, this validation step guarantees that the mutation request contains the information required for downstream governance evaluation.

The mutation schema defines:

- the mutation type being requested
- the required attributes that must be present in the mutation payload
- optional attributes that may appear in the payload
- the permitted data types and structures associated with each field

During schema evaluation, the governance control plane performs several checks.

First, the control plane verifies that the mutation type declared in the request corresponds to a mutation type defined within the specification. In the identity provisioning example, the allowed mutation type is `identity.create`. Mutation types not declared within the specification are rejected.

Second, the control plane verifies that all required fields defined in the schema are present within the mutation payload. Missing attributes indicate that the mutation request is incomplete and therefore cannot be evaluated against governance rules.

Third, the control plane validates the data structure and format of each field to ensure that the mutation payload conforms to the canonical representation expected by the governance pipeline.

If the mutation request satisfies the schema requirements, the control plane constructs a canonical mutation representation that will be used during the subsequent governance evaluation stages.

The canonical mutation representation is embedded within, and normalized with respect to, the governance action envelope structure defined in Appendix A.

A simplified canonical mutation envelope may appear as follows:

```
canonical_mutation:

mutation_id: 84b2c0a7
mutation_type: identity.create
agent_spec_id: PBSAI.ICAM.C1.v1
execution_target: identity_management_system

attributes:
employee_id: 483920
department: finance
manager_id: 77341
job_code: FIN-ANL-02
```

Canonicalization ensures that all mutation requests entering the governance pipeline share a consistent internal representation. This normalization step is important for deterministic policy evaluation and simplifies downstream rule processing.

The canonical mutation representation therefore becomes the input to the governance rule evaluation stage.

6.3 Governance Rule Evaluation

Once the mutation request has been canonicalized, the governance control plane evaluates the mutation against the compiled rule sets contained within the agent operating specification. Evaluation includes validation of the execution target to ensure that the proposed mutation is authorized for execution within the designated system context defined by the agent operating specification.

The purpose of this stage is to determine whether the proposed operational mutation complies with the governance constraints derived from external policy sources.

The governance rule evaluation stage processes the canonical mutation using the compiled rule definitions described earlier in Section 4.5. Each rule is evaluated against the mutation attributes and contextual metadata associated with the request.

A simplified representation of the rule evaluation process appears below.

```
rule_evaluation:
```

```
rule: verify_authoritative_source
input:
event_source: HRIS
result: PASS

rule: verify_required_attributes
input:
employee_id: 483920
department: finance
manager_id: 77341
result: PASS

rule: verify_role_mapping
input:
role_mapping_source: IDM
result: PASS

rule: segregation_of_duties_check
input:
entitlements: [finance_read, expense_submit]
result: PASS
```

Each rule produces a deterministic evaluation outcome indicating whether the mutation satisfies the associated governance constraint.

If any rule produces a failure condition, the mutation request is rejected and an authorization denial is returned to the requesting agent. If all governance rules evaluate successfully, the mutation request proceeds to the commit authorization stage.

The use of deterministic rule evaluation ensures that governance decisions remain reproducible across multiple executions of the pipeline. Because the same compiled policy rules are applied to the same canonical mutation representation, identical input conditions will produce identical governance outcomes.

Deterministic evaluation is an important requirement for governance systems operating within distributed environments where multiple components may rely on consistent authorization decisions. Distributed systems research has long recognized the importance of deterministic decision processes in maintaining consistent system behavior across distributed nodes (Lamport, 1978; Ongaro and Ousterhout, 2014).

6.4 Commit Authorization

If the mutation request successfully satisfies all governance rule evaluations, the governance control plane proceeds to the commit authorization stage. The purpose of this stage is to produce a final authorization decision that allows the operational mutation to be executed by the designated execution target identified in the authorized mutation. Such constraints correspond to explicit authorization gates in governance control planes, where execution may be suspended pending human approval (Willis, 2026a).

Importantly, the governance control plane does not perform the operational mutation itself. Instead, it produces a governance decision indicating whether the mutation is authorized under the compiled governance specification and whether it may be routed to the designated execution target. The requesting agent remains responsible for proposing the mutation, but the operational state change is performed by the authorized execution target rather than by the proposing agent itself.

A simplified authorization response generated by the governance control plane may appear as follows:

```
governance_decision:

mutation_id: 84b2c0a7
decision: APPROVED
evaluated_spec: PBSAI.ICAM.C1.v1
execution_target: identity_management_system
evaluation_timestamp: 2026-03-15T10:22:14Z
```

If the decision is `APPROVED`, the authorized mutation may be routed to the designated execution target for application within the appropriate enterprise system. In the identity provisioning scenario examined in this paper, this may correspond to transmitting the approved mutation to the enterprise identity management platform, which then performs creation of the new user identity.

In cases where one or more governance rules fail during evaluation, the governance control plane instead produces a denial response indicating the reason for the rejection. The agent may then either terminate the operation or initiate remediation actions depending on its internal logic.

The commit authorization stage therefore serves as the final governance checkpoint prior to operational execution. This model is conceptually similar to transaction commit semantics used in distributed system architectures, where a transaction must pass a series of validation stages before it can be committed to the system state (Gray and Reuter, 1993).

6.5 Audit and Lifecycle Recording

The final stage of the governance pipeline records the complete lifecycle of the evaluated mutation request. This stage ensures that governance decisions remain auditable and that the provenance of operational actions can be reconstructed for compliance and security analysis.

The governance control plane records multiple artifacts associated with the mutation request, including:

- the original mutation request submitted by the agent
- the canonical mutation representation constructed during schema evaluation
- the results of each governance rule evaluation
- the final authorization decision
- timestamps and contextual metadata associated with the evaluation

A simplified representation of an audit record may appear as follows:

```
audit_record:

mutation_id: 84b2c0a7
agent_id: C1_identity_provisioning_agent
agent_spec: PBSAI.ICAM.C1.v1
event_source: HRIS

execution_target: identity_management_system
```

```
rule_results:
verify_authoritative_source: PASS
verify_required_attributes: PASS
verify_role_mapping: PASS
segregation_of_duties_check: PASS

decision: APPROVED

evaluation_timestamp: 2026-03-15T10:22:14Z
dispatch_timestamp: 2026-03-15T10:22:15Z

execution_status: SUCCESS
execution_timestamp: 2026-03-15T10:22:18Z
```

The audit record captures both the authorization decision and the subsequent execution outcome, enabling full lifecycle reconstruction of the mutation from proposal through execution.

This ensures that all actions are attributable, verifiable, and reconstructible from canonical system state and evidence lineage, consistent with governance control plane requirements (Willis, 2026a).

Maintaining complete records of governance evaluation enables organizations to demonstrate compliance with security policies and regulatory requirements governing identity lifecycle operations. These audit records also support forensic analysis in cases where operational actions must be investigated after the fact.

The importance of maintaining verifiable provenance for data transformations and operational actions has been widely studied in database and information systems research. Provenance frameworks allow systems to record the lineage of data and operations in order to reconstruct how a particular state was produced (Buneman et al., 2001; Cheney et al., 2009).

Within the governance control plane architecture, the audit record therefore functions as a provenance artifact describing how the operational mutation was evaluated and authorized.

By recording each stage of the governance pipeline, the system ensures that governance decisions remain transparent, reproducible, and accountable across the enterprise automation environment.

The next section examines how this deterministic governance execution model enables reproducible governance behavior across distributed autonomous systems.

7 Deterministic Governance Properties

The governance control plane architecture described in the previous sections is designed to ensure that governance decisions remain deterministic, reproducible, and auditable across distributed autonomous systems. Deterministic governance behavior is essential for enterprise automation environments in which multiple agents may interact with shared enterprise infrastructure.

Traditional governance systems often rely on human interpretation of policy or loosely defined procedural controls. In contrast, the governance control plane model transforms external governance policy into structured machine-interpretable artifacts through the governance compilation process described earlier in this paper. These compiled artifacts are then evaluated through a deterministic execution pipeline that produces consistent governance decisions for identical input conditions.

The deterministic behavior of the governance control plane arises from three architectural characteristics:

- governance policies are compiled into structured specifications prior to runtime

- mutation requests are normalized into canonical representations before evaluation
- rule evaluation is performed using deterministic rule sets

Together, these properties ensure that the governance evaluation pipeline behaves as a deterministic state transition system in which the same input conditions always produce the same governance outcome.

7.1 Deterministic Policy Evaluation

Deterministic policy evaluation begins during the governance compilation phase. As described earlier in Section 3, governance compilation translates external policy sources into a structured agent operating specification. The compiled specification contains the governance rules, event source definitions, and mutation schema required for runtime evaluation.

In practice, governance compilation is typically performed by a human policy engineer or system architect who interprets external governance policy and encodes the resulting constraints into the specification. This process may be supported by tooling or generative AI systems that assist in analyzing policy documents and identifying candidate governance constraints. However, the authoritative interpretation of governance policy ultimately reflects deliberate human decisions about how policy should be implemented within the enterprise system architecture.

The resulting specification is stored within the governance control plane as a versioned policy artifact. These specifications are typically maintained within a policy registry that functions as the authoritative repository for compiled governance specifications.

A simplified example of a policy registry entry may appear as follows:

```
policy_registry:

specification_id: PBSAI.ICAM.C1.v1
domain: identity_governance
version: 1.0
compilation_timestamp: 2026-02-15
compiled_by: governance_engineer
```

The policy registry ensures that governance evaluation always occurs against a stable and versioned specification. When an autonomous agent submits a mutation request, the request references the specification identifier under which it is operating. The governance control plane retrieves the corresponding compiled specification from the registry and evaluates the mutation request through the governance pipeline described earlier in Section 6.

Because the compiled specification remains fixed during evaluation, the governance control plane produces consistent authorization decisions for identical mutation requests. This property ensures that governance decisions remain predictable and reproducible even when multiple autonomous agents operate concurrently within the enterprise environment.

7.2 Reproducibility of Governance Decisions

Reproducibility is a critical property for governance systems operating within distributed enterprise infrastructures. Organizations must be able to demonstrate that governance decisions were made consistently and in accordance with defined policy frameworks.

The governance control plane achieves reproducibility through the combination of canonical mutation representations and versioned policy specifications. Each mutation request is normalized into a canonical

form before evaluation, and each evaluation is performed against a specific version of the compiled governance specification.

This design ensures that governance decisions can be reproduced at a later time by re-evaluating the same canonical mutation against the same policy specification. The audit records generated during the governance pipeline therefore provide a complete record of the inputs and rules used to produce a particular governance decision.

Maintaining deterministic evaluation across distributed system components has long been recognized as an important design principle in distributed computing architectures. Systems that rely on deterministic processing models are able to maintain consistent behavior across distributed nodes even when components operate asynchronously (Lamport, 1978; Kleppmann, 2017).

By structuring governance evaluation as a deterministic pipeline operating on canonical mutation representations, the governance control plane ensures that authorization decisions remain stable and reproducible across large-scale enterprise automation environments.

7.3 Distributed System Consistency Considerations

Enterprise automation environments frequently operate across large-scale distributed system infrastructures in which multiple agents and services interact concurrently with shared enterprise resources. In such environments, governance enforcement mechanisms must ensure that authorization decisions remain consistent even when mutation requests are generated from different system components operating asynchronously.

The deterministic governance control plane architecture addresses this requirement by separating governance evaluation from operational execution. Mutation requests generated by autonomous agents are evaluated centrally within the governance control plane before operational actions are executed within enterprise systems.

This design reduces the likelihood of inconsistent authorization decisions arising from divergent local policy interpretations. Rather than allowing individual agents to independently interpret governance rules, all mutation requests are evaluated against the same compiled policy specification maintained within the governance control plane.

Consistency considerations in distributed systems have been extensively studied in the context of distributed data stores and control-plane architectures. Brewer's CAP theorem highlights the trade-offs between consistency, availability, and partition tolerance in distributed environments (Brewer, 2012). Governance control plane architectures typically prioritize consistent policy evaluation over maximum availability, since inconsistent authorization decisions could produce security violations or policy non-compliance.

Similarly, distributed coordination systems often rely on consensus algorithms to ensure that shared state remains consistent across distributed nodes. Algorithms such as Paxos and Raft allow distributed components to maintain agreement on system state even in the presence of failures or network partitions (Lamport, 1998; Ongaro and Ousterhout, 2014). While the governance control plane described in this paper does not require consensus for individual rule evaluations, similar distributed coordination mechanisms may be used to maintain the integrity of the policy registry and governance audit records.

Modern distributed infrastructure platforms frequently implement centralized control-plane architectures that coordinate the behavior of large numbers of distributed components. Examples include software-defined networking control planes and container orchestration systems such as Kubernetes, which maintain centralized system state and distribute control decisions across distributed worker nodes (Kreutz et al., 2015; Burns et al., 2016).

The governance control plane follows a similar architectural pattern. Autonomous agents act as distributed execution nodes that propose operational mutations, while the governance control plane evaluates those mutations against centrally managed policy specifications.

7.4 Governance Control Planes and Distributed Coordination

The concept of a governance control plane is closely related to control-plane architectures used in modern distributed infrastructure systems. In such architectures, the control plane maintains the authoritative system state and enforces system-wide policies, while the data plane performs operational execution.

In the context of enterprise automation systems, the governance control plane acts as the authoritative policy evaluation layer responsible for determining whether operational mutations proposed by autonomous agents are permitted under enterprise governance policy.

This architecture parallels the separation between control-plane and data-plane functionality used in software-defined networking and large-scale distributed orchestration systems. In these systems, the control plane maintains the desired state of the system and enforces policies governing network configuration or infrastructure management (Kreutz et al., 2015).

Similarly, large-scale distributed databases maintain centralized coordination services to ensure consistent system behavior. Systems such as ZooKeeper provide coordination primitives used by distributed services to maintain shared configuration state and perform distributed synchronization (Hunt et al., 2010). Globally distributed database systems such as Google's Spanner rely on coordinated state management and global time synchronization to maintain consistent transaction ordering across geographically distributed infrastructure (Corbett et al., 2013).

The governance control plane architecture described in this paper extends these control-plane design principles to the domain of enterprise governance enforcement for autonomous agents. By centralizing governance evaluation and maintaining compiled policy specifications within a versioned registry, the architecture ensures that governance decisions remain consistent across large-scale automation environments.

This approach enables organizations to deploy autonomous agents across distributed enterprise systems while preserving centralized governance oversight. Agents remain free to perform complex reasoning and automation tasks, but any operational mutation affecting enterprise resources must pass through the governance control plane before execution.

The next section discusses the implications of this architecture for governance enforcement in autonomous AI systems and examines how deterministic governance control planes can support safe and accountable enterprise automation.

8 Implications for Autonomous System Governance

The increasing deployment of autonomous agents across enterprise environments introduces new challenges for governance, security, and accountability. Modern AI systems are increasingly capable of performing complex reasoning, planning, and decision-making tasks, which allows them to automate operational processes that were previously performed by humans.

While these capabilities offer significant benefits for enterprise productivity and system management, they also introduce risks if autonomous systems are permitted to perform operational actions without appropriate governance controls. Several recent studies have highlighted the potential risks associated with autonomous AI systems that operate without adequate oversight mechanisms, including unintended system behavior, privilege escalation, and the misuse of AI capabilities in sensitive operational contexts (Amodei et al., 2016; Brundage et al., 2018; Hendrycks et al., 2022).

Traditional governance models rely heavily on human oversight and procedural controls to manage operational risk. However, such models become increasingly difficult to apply in environments where autonomous systems operate at machine speed and may generate large volumes of operational actions across distributed infrastructure.

The governance control plane architecture described in this paper addresses this challenge by introducing a deterministic governance enforcement layer that operates independently of agent reasoning. Rather than

attempting to constrain how autonomous agents reason about operational tasks, the architecture constrains which operational mutations may ultimately be executed within the enterprise system.

This design approach reflects a broader trend in AI safety research that emphasizes the importance of maintaining reliable control mechanisms over the operational behavior of autonomous systems (Russell, 2019). By separating reasoning from action authorization, the governance control plane ensures that even highly capable agents remain subject to enforceable governance constraints.

8.1 Separation of Reasoning and Action Authorization

A central architectural principle underlying the governance control plane model is the separation between agent reasoning and action authorization. Autonomous agents may employ complex reasoning processes internally, including planning algorithms, large language model reasoning chains, or reinforcement learning strategies. However, these reasoning processes do not directly determine whether operational actions are executed.

Instead, the agent must submit proposed operational mutations to the governance control plane, which evaluates those mutations against compiled governance policy before authorizing execution.

This architectural separation is consistent with recent research on agent architectures that combine reasoning and action generation within AI systems. Frameworks such as ReAct and Reflexion illustrate how language-model-based agents can interleave reasoning and action generation during task execution (Yao et al., 2023; Shinn et al., 2023). While these approaches enable agents to perform complex interactive tasks, they also highlight the need for external governance mechanisms capable of controlling the operational actions produced by such systems.

Similarly, research examining the societal and ethical risks associated with large-scale AI systems has emphasized the importance of governance mechanisms capable of constraining the behavior of autonomous systems deployed in real-world environments (Bommasani et al., 2021; Weidinger et al., 2022).

The governance control plane architecture complements these research directions by providing a concrete systems architecture through which governance constraints can be enforced at runtime. Rather than relying solely on model alignment techniques or training-time safeguards, the architecture introduces an execution-layer governance mechanism that operates independently of the agent's internal reasoning processes.

This design ensures that even if the internal reasoning mechanisms of an autonomous agent evolve or change over time, the governance constraints governing operational behavior remain enforceable through the external control plane.

8.2 Governance Control Planes as Infrastructure for Safe AI Systems

As autonomous systems continue to expand their role within enterprise environments, the need for reliable governance infrastructure becomes increasingly important. Traditional governance mechanisms often rely on human review processes, manual policy enforcement, or loosely coupled monitoring systems. These approaches may be insufficient for environments in which autonomous systems generate operational actions at machine speed.

The governance control plane architecture described in this paper introduces a systems-level approach to governance enforcement. By compiling governance policy into structured specifications and evaluating operational mutations through a deterministic execution pipeline, the architecture provides a scalable mechanism for enforcing governance constraints across large numbers of autonomous agents.

This approach mirrors the evolution of infrastructure control planes in modern distributed systems. In software-defined networking, container orchestration platforms, and distributed databases, centralized control planes maintain system-wide policy and coordination state while distributed components perform op-

erational execution. Applying similar architectural principles to governance enforcement allows enterprise systems to maintain consistent governance behavior even as automation capabilities expand.

The governance control plane therefore functions as foundational infrastructure for safe enterprise AI deployment. Autonomous agents remain capable of performing sophisticated reasoning and automation tasks, but any operational mutation affecting enterprise resources must pass through a deterministic governance evaluation process before execution.

This architecture provides a practical mechanism for implementing execution-layer governance for AI systems operating within real-world enterprise environments.

9 Conclusion

This paper presented a deterministic governance control plane architecture designed to enforce enterprise governance policy for autonomous systems operating within distributed enterprise environments. The architecture separates agent reasoning from action authorization by introducing an independent governance evaluation pipeline that determines whether operational mutations proposed by autonomous agents may be executed.

The governance control plane evaluates mutation requests against compiled governance specifications derived from external policy sources. Through the governance compilation process, organizational policies, regulatory requirements, and identity governance frameworks are translated into structured specifications that can be evaluated deterministically at runtime.

Mutation requests submitted by autonomous agents are evaluated through a five-stage governance pipeline consisting of identity and context validation, schema evaluation, rule evaluation, commit authorization, and audit recording. This structured pipeline ensures that governance decisions remain predictable, reproducible, and auditable across distributed enterprise systems.

By centralizing governance evaluation within a dedicated control plane, the architecture ensures that operational actions performed by autonomous agents remain aligned with enterprise governance policy regardless of how the agents internally reason about tasks. This design enables organizations to deploy increasingly capable automation systems while maintaining consistent governance oversight.

As autonomous AI systems become more prevalent within enterprise environments, governance control planes may become a critical architectural component for ensuring safe, accountable, and policy-compliant automation.

Copyright

Code and Specification Availability

The Artificial Intelligence Governance Control Plane (AGCP) specification and related artifacts referenced in this paper are publicly available.

AGCP Specification (archived release):
https://doi.org/10.5281/zenodo.18923302
AGCP Reference Repository:
https://github.com/jwillisSFT/agcp-spec

Patent Notice

Elements of the architecture described in this work may be covered by pending patent applications. Publication of this paper does not grant a license to any patent rights associated with the described systems or methods. The author reserves all patent rights associated with the described systems and methods.

References

Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., and Mané, D. Concrete Problems in AI Safety. arXiv, 2016. https://arxiv.org/abs/1606.06565

Bishop, M., and Dilger, M. Checking for Race Conditions in File Accesses. Computing Systems, 9(2):131–152, 1996. https://www.usenix.org/legacy/publications/compsystems/1996/spr_bishop.pdf

Bommasani, R., Hudson, D., Adeli, E., Altman, R., Arora, S., von Arx, S., et al. On the Opportunities and Risks of Foundation Models. arXiv, 2021. https://arxiv.org/abs/2108.07258

Brewer, E. CAP Twelve Years Later: How the "Rules" Have Changed. Computer, 45(2):23–29, 2012. https://doi.org/10.1109/MC.2012.37

Brundage, M., Avin, S., Clark, J., Toner, H., Eckersley, P., Garfinkel, B., et al. The Malicious Use of Artificial Intelligence: Forecasting, Prevention, and Mitigation. arXiv, 2018. https://arxiv.org/abs/1802.07228

Buneman, P., Khanna, S., and Tan, W. Why and Where: A Characterization of Data Provenance. Proceedings of the International Conference on Database Theory, 2001. https://doi.org/10.1007/3-540-44503-X_20

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., and Wilkes, J. Borg, Omega, and Kubernetes. Communications of the ACM, 59(5):50–57, 2016. https://doi.org/10.1145/2890784

Cheney, J., Chiticariu, L., and Tan, W. Provenance in Databases: Why, How, and Where. Foundations and Trends in Databases, 2009. https://doi.org/10.1561/1900000006

Corbett, J., Dean, J., Epstein, M., Fikes, A., Frost, C., Furman, J., et al. Spanner: Google's Globally Distributed Database. ACM Transactions on Computer Systems, 2013. https://doi.org/10.1145/2491245

Ferraiolo, D., Sandhu, R., Gavrila, S., Kuhn, D., and Chandramouli, R. Proposed NIST Standard for Role-Based Access Control. ACM Transactions on Information and System Security, 2001. https://doi.org/10.1145/501978.501980

Gray, J., and Reuter, A. Transaction Processing: Concepts and Techniques. Morgan Kaufmann, 1993.

Hendrycks, D., Mazeika, M., and Woodside, T. Unsolved Problems in ML Safety. arXiv, 2022. https://arxiv.org/abs/2109.13916

Hunt, P., Konar, M., Junqueira, F., and Reed, B. ZooKeeper: Wait-Free Coordination for Internet-Scale Systems. USENIX Annual Technical Conference, 2010. https://www.usenix.org/legacy/event/atc10/tech/full_papers/Hunt.pdf

Kleppmann, M. Designing Data-Intensive Applications. O'Reilly Media, 2017.

Kreutz, D., Ramos, F., Verissimo, P., Rothenberg, C., Azodolmolky, S., and Uhlig, S. Software-Defined Networking: A Comprehensive Survey. Proceedings of the IEEE, 2015. https://doi.org/10.1109/JPROC.2014.2371999

Lamport, L. Time, Clocks, and the Ordering of Events in a Distributed System. Communications of the ACM, 1978. https://doi.org/10.1145/359545.359563

Lamport, L. The Part-Time Parliament. ACM Transactions on Computer Systems, 1998. https://doi.org/10.1145/279227.279229

Lampson, B.W. Protection. ACM SIGOPS Operating Systems Review, 8(1):18–24, 1974. https://doi.org/10.1145/775265.775268

National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework (AI RMF 1.0). 2023. https://www.nist.gov/itl/ai-risk-management-framework

National Institute of Standards and Technology. Digital Identity Guidelines (SP 800-63). https://pages.nist.gov/800-63-3/

National Institute of Standards and Technology. Security and Privacy Controls for Information Systems and Organizations (SP 800-53 Rev.5). https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final

Ongaro, D., and Ousterhout, J. In Search of an Understandable Consensus Algorithm (Raft). USENIX ATC, 2014. https://raft.github.io/raft.pdf

Park, J., O'Brien, J., Cai, C., Morris, M., Liang, P., and Bernstein, M. Generative Agents: Interactive Simulacra of Human Behavior. arXiv, 2023. https://arxiv.org/abs/2304.03442

Russell, S. Human Compatible: Artificial Intelligence and the Problem of Control. Viking, 2019.

Saltzer, J., and Schroeder, M. The Protection of Information in Computer Systems. Proceedings of the IEEE, 1975. https://doi.org/10.1109/PROC.1975.9939

Sandhu, R., Coyne, E., Feinstein, H., and Youman, C. Role-Based Access Control Models. IEEE Computer, 1996. https://doi.org/10.1109/2.485845

Schneider, F. Enforceable Security Policies. ACM Transactions on Information and System Security, 2000. https://doi.org/10.1145/353323.353382

Shinn, N., Labash, B., and Gopinath, A. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv, 2023. https://arxiv.org/abs/2303.11366

Weidinger, L., Uesato, J., Rauh, M., Griffin, C., Huang, P., Mellor, J., et al. Ethical and Social Risks of Large Language Models. arXiv, 2022. https://arxiv.org/abs/2112.04359

Willis, J.M. *Artificial Intelligence Governance Control Plane (AGCP) Specification v0.9.0*. Sustainable Future Tech Publishing, 2026. https://doi.org/10.5281/zenodo.18923302. Live repository: https://github.com/jwillisSFT/agcp-spec

Willis, J.M. *The PBSAI Governance Ecosystem: A Multi-Agent AI Reference Architecture for Securing Enterprise AI Estates*. arXiv, 2026. https://arxiv.org/pdf/2602.11301

Willis, J.M. *Runtime Governance Architecture: Consistent Governance Execution for Enterprise Systems and Autonomous Agents*. arXiv, 2026 (under review).

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., et al. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv, 2023. https://arxiv.org/abs/2210.03629

Appendix A Canonical Governance Action Envelope Specification

This appendix defines the canonical structure of governance interaction artifacts exchanged between autonomous agents and the AI Governance Control Plane (AGCP). These artifacts represent candidate operational mutations submitted by governed agents for deterministic evaluation by the governance control plane.

The governance action envelope provides a standardized representation of proposed operational mutations, including mutation attributes, supporting evidence, and provenance metadata. The envelope structure allows the AGCP to evaluate mutation proposals using policy registries, governance rules, and deterministic evaluation pipelines.

The envelope specification defined in this appendix serves as the canonical interface between governed agents and the AGCP.

Normative Language

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this appendix are to be interpreted as describing requirement levels for implementations of governed agents and governance control plane systems.

Appendix A.1 Governance Action Envelope Overview

The governance action envelope is the primary interaction artifact used by governed agents to submit candidate operational mutations to the AI Governance Control Plane.

Every operational mutation proposed by an agent MUST be encapsulated within a governance action envelope prior to submission to the AGCP.

The envelope contains the following categories of information:

- agent identity
- mutation description
- policy context
- evidence artifacts
- provenance metadata
- submission metadata

The envelope provides the information necessary for the governance control plane to evaluate whether the mutation is permitted under the applicable governance policies.

Appendix A.2 Canonical Envelope Structure

Governance action envelopes MUST conform to a canonical structural representation. The envelope structure ensures that mutation proposals can be interpreted consistently by the governance control plane.

The canonical envelope structure is defined as follows:

```
governance_action_envelope

agent_identity
mutation_type
mutation_attributes
policy_context
evidence
provenance
submission_metadata
signature
```

Each component of the envelope is described in the following sections.

Appendix A.3 Agent Identity

The agent identity field identifies the agent submitting the mutation proposal.

The agent identity section MUST include the following information:

- agent identifier
- issuing authority
- trust domain
- credential reference

This information allows the governance control plane to verify the identity and authority scope of the submitting agent.

```
agent_identity

agent_id
issuing_authority
trust_domain
credential_reference
```

The AGCP MUST verify the agent identity before evaluating the mutation proposal.

Appendix A.4 Mutation Structure

The mutation structure describes the operational change proposed by the agent and the execution context in which that change is intended to be applied. Every governance action envelope MUST contain a mutation description that conforms to a canonical schema recognized by the AI Governance Control Plane.

Mutation schemas define the attributes associated with specific mutation types. The mutation structure MUST contain a mutation type identifier and the attributes associated with the mutation.

The canonical mutation representation is defined as follows:

```
mutation_type
execution_target
mutation_attributes
```

The `execution_target` field identifies the system, service, or component authorized to apply the governed operational mutation. This field is part of the canonical mutation representation because authorization decisions are bound not only to the mutation type and attributes, but also to the execution context in which the mutation is to be applied.

The mutation_type field identifies the category of operational mutation being proposed. Examples include identity provisioning, infrastructure modification, configuration update, or policy modification.

The mutation_attributes field contains the parameters required to execute the mutation if authorized.

Example mutation structure:

```
mutation_type:
identity_account_provisioning
execution_target: enterprise_directory
mutation_attributes:
user_identifier: jsmith
account_domain: enterprise_directory
role_assignment: analyst
access_scope: data_platform
```

Mutation attributes MUST conform to the schema definitions recognized by the governance control plane.

The AGCP MUST validate the mutation structure prior to performing governance rule evaluation.

Appendix A.5 Policy Context

The policy context section provides governance information relevant to the proposed mutation.

Policy context allows the governance control plane to evaluate mutation proposals within the appropriate policy environment.

The policy context representation is defined as follows:

```
policy_context

policy_identifiers
policy_domain
operational_environment
risk_classification
```

The policy context MAY reference one or more governance policies that are applicable to the mutation proposal.

The AGCP MAY use the policy context to determine which governance rule sets should be applied during evaluation.

Appendix A.6 Evidence Representation

Evidence artifacts provide the supporting information required for governance evaluation.

Evidence allows the governance control plane to determine whether the mutation proposal is supported by authoritative lifecycle events or operational conditions.

The evidence structure is defined as follows:

```
evidence

evidence_source
evidence_type
evidence_reference
evidence_description
```

Evidence artifacts MAY include references to:

- lifecycle events
- identity system records
- infrastructure monitoring events
- workflow transitions
- policy triggers

Example evidence representation:

```
evidence:

evidence_source: HRIS
evidence_type: lifecycle_event
evidence_reference: HR_EVENT_2026_03_15_0021
evidence_description: new employee onboarding event
```

The governance control plane SHOULD verify the authenticity and integrity of evidence artifacts when possible.

Appendix A.7 Provenance Metadata

Provenance metadata describes the origin of the mutation proposal and the context in which the proposal was generated.

Provenance information supports governance auditing and operational traceability.

The provenance structure is defined as follows:

```
provenance

generation_timestamp
agent_runtime_identifier
source_event_reference
generation_context
```

Provenance metadata MAY include references to system state, agent runtime configuration, or operational events that contributed to the generation of the mutation proposal.

Example provenance metadata:

```
provenance:

generation_timestamp: 2026-03-15T10:30:21Z
agent_runtime_identifier: identity_provisioning_agent_c1
source_event_reference: HR_EVENT_2026_03_15_0021
generation_context: onboarding workflow
```

The AGCP MAY record provenance metadata as part of the governance audit record.

Appendix A.8 Submission Metadata

Submission metadata provides operational information describing when and how the envelope was submitted.

Submission metadata is defined as follows:

```
submission_metadata

submission_timestamp
envelope_identifier
submission_channel
```

The envelope identifier MUST uniquely identify the submitted governance action envelope.

Appendix A.9 Authorization Response Structure

Following governance evaluation, the AGCP returns an authorization response describing the outcome of the governance decision.

The authorization response structure is defined as follows:

```
authorization_response

decision
decision_reason
policy_reference
authorization_token
decision_timestamp
```

The decision field indicates whether the mutation proposal has been authorized or rejected.

Possible decision values include:

- authorized
- rejected
- deferred

If the mutation is authorized, the AGCP MAY issue an authorization token that permits execution of the mutation within operational systems.

Example authorization response:

```
authorization_response:

decision: authorized
decision_reason: lifecycle event validated
policy_reference: IDENTITY_GOV_POLICY_002
authorization_token: AGCP_AUTH_2026_03_15_4472
decision_timestamp: 2026-03-15T10:30:24Z
```

Operational systems MUST verify the authorization token before executing any mutation derived from a governance action envelope.